ThinkingRoot — The Open-Source Knowledge Compiler for AI Agents

Design Spec v1.0 Date: 2026-04-08 **Author:** Naveen + Claude **Status:** Name finalized, ready for implementation plan

---

Table of Contents

1. Product Identity
2. Problem Statement
3. Core Architecture — The Compilation Pipeline
4. Data Model
5. Multi-Agent Safety System
6. Belief Revision Engine
7. Access Control Model
8. Product Layers & Open-Source Boundary
9. Pricing Model
10. Competitive Positioning
11. Go-to-Market
    - 11.1 Personal Use Case — Second Brain Mode
12. Technical Stack
13. Repository Structure
14. Build Sequence
15. Success Metrics
16. Risks & Mitigations

---

1. Product Identity

Name: ThinkingRoot

Why "ThinkingRoot": The root of all thinking and knowledge — the foundational layer where your organization's knowledge is grounded, verified, and compiled. "Root" suggests both the foundation (root of a tree) and root-level access (complete visibility). The full brand is "ThinkingRoot" but the CLI is simply `root` — fast to type, memorable, essential.

CLI: `root` (4 letters, zero friction) **Package names:** `thinkingroot` (crates.io, PyPI, npm)

Category: Knowledge Compiler

One-liner: "The open-source knowledge compiler for AI agents."

Longer pitch: "ThinkingRoot compiles your docs, code, chats, and tickets into verified, linked knowledge that agents read in 2K tokens instead of 50K. It runs continuously — like CI/CD for what your organization knows."

Dinner party version: "You know how code has compilers? We built ThinkingRoot — a compiler for knowledge. AI agents stop re-reading everything and just read the compiled version."

Core framing: Compilation, not memory.

---

2. Problem Statement

AI agents and teams work with knowledge that is:

- **Distributed** across docs, repos, chats, tickets, PDFs, meetings
- **Constantly changing** with no mechanism to detect staleness
- **Contradictory** across sources with no resolution
- **Re-read from scratch** every session, wasting tokens, time, and money
- **Unverified** — agents trust their memory blindly with no provenance
- **Unsafe** — persistent memory is a serious poisoning surface (eTAMP, Zombie Agents papers, April 2026)

Current solutions (Mem0, Zep, Supermemory, Letta, Cognee, LangMem) all treat this as a **retrieval** problem — store and search. None treat it as a **compilation** problem — transform raw knowledge into optimized, verified, source-linked output.

The gap: Nobody does ingest → compile → verify → serve → maintain as an end-to-end pipeline.

Target Users (5 types)

User Type	Pain Point	Tier	Entry Point
Personal / second brain	Notes scattered across apps, no connections, nothing compiled	Free (CLI)	`root add ./notes && root compile`
Solo developers	Agents re-read everything, waste tokens	Free → Pro	CLI → MCP server for their agent
AI agent builders	Multi-agent products need shared memory	Pro → Team	Python SDK integration
Engineering teams	Docs stale, decisions lost in Slack, onboarding painful	Team	Dashboard + connectors
Enterprise platform teams	100+ agents, compliance, safety, audit trails	Enterprise	Full platform deployment

---

3. Core Architecture

ThinkingRoot is a 6-stage compilation pipeline:

```
┌─────┐ → ┌───────┐ → ┌────┐ → ┌───────┐ → ┌──────┐ → ┌─────┐
│PARSE│   │EXTRACT│   │LINK│   │COMPILE│   │VERIFY│   │SERVE│
└─────┘   └───────┘   └────┘   └───────┘   └──────┘   └─────┘
```

Stage 1: PARSE (Rust)

Converts raw data into a normalized Intermediate Representation (IR).

Phase 1 sources: Markdown, code repos (tree-sitter), GitHub issues/PRs, Slack/chat messages, PDFs, web pages.

```rust
struct DocumentIR {
    source: SourceRef,
    timestamp: DateTime,
    author: Option<Author>,
    content_type: ContentType,
    chunks: Vec<Chunk>,
    raw_hash: Hash,
}
```

Stage 2: EXTRACT (Rust + LLM)

Extracts structured knowledge from the IR: Claims, Entities, and Relations.

- **Claims** — atomic, source-locked, typed, timestamped statements
- **Entities** — people, systems, concepts with aliases
- **Relations** — typed connections between entities with evidence

Stage 3: LINK (Rust graph engine)

Builds the temporal knowledge graph:

- Entity resolution (alias merging)
- Contradiction detection
- Temporal ordering
- Cluster detection

Stage 4: COMPILE (Rust + LLM)

Transforms the linked graph into optimized knowledge representations:

Compilation Target	Consumer
Task Pack	Coding agents
Entity Page	Humans + agents
Decision Log	Humans
Architecture Map	Humans + agents
Runbook	Humans
Contradiction Report	Humans

Compilation Target	Consumer
Agent Brief	Agents

Compilation = compression + deduplication + conflict resolution + citation.

Stage 5: VERIFY (Memory CI)

Continuous verification, like GitHub Actions for knowledge:

Check	What it catches
Staleness	Claims older than TTL with no refresh
Contradiction	Two active claims that conflict
Orphan	Claims whose source was deleted
Confidence decay	Claims with superseded sources
Poisoning	Claims from untrusted/anomalous sources
Leakage	Claims violating sensitivity policies
Coverage	Entities with too few claims

Produces a **Knowledge Health Score** with freshness, consistency, coverage, safety, and provenance sub-scores.

Stage 6: SERVE

- Python SDK via PyO3
- REST API (Axum)
- MCP Server (Model Context Protocol)
- CLI
- Artifact files (markdown to disk/git)
- Webhooks

Key architectural property: Incremental compilation. When one document changes, only affected claims, entities, and artifacts recompile. Like `make` — not a full rebuild.

---

4. Data Model

Seven core types:

4.1 Source

```
struct Source {
    id: SourceId,
    uri: String,                // file path, URL, git commit, message ID
    source_type: SourceType,    // Git, Document, Chat, API, Manual
    author: Option<String>,
```

```
    created_at: DateTime,
    content_hash: Hash,
    trust_level: TrustLevel,    // Verified, Trusted, Unknown, Untrusted,
Quarantined
}
```

4.2 Claim (fundamental unit)

```
struct Claim {
    id: ClaimId,
    statement: String,
    claim_type: ClaimType,      // Fact, Decision, Opinion, Plan,
Requirement, Metric
    source: SourceId,
    source_span: Option<Span>,
    confidence: f64,
    valid_from: DateTime,
    valid_until: Option<DateTime>,
    sensitivity: Sensitivity,   // Public, Internal, Confidential,
Restricted
    workspace: WorkspaceId,
    extracted_by: PipelineVersion,
    superseded_by: Option<ClaimId>,
}
```

4.3 Entity

```
struct Entity {
    id: EntityId,
    canonical_name: String,
    entity_type: EntityType,    // Person, System, Service, Concept, Team,
API, Database
    aliases: Vec<String>,
    attributes: Vec<ClaimId>,
    first_seen: DateTime,
    last_updated: DateTime,
}
```

4.4 Relation

```
struct Relation {
    id: RelationId,
    from: EntityId,
    to: EntityId,
    relation_type: RelationType, // DependsOn, OwnedBy, Replaces,
Contradicts, Implements
```

```
    evidence: Vec<ClaimId>,
    strength: f64,
    valid_from: DateTime,
    valid_until: Option<DateTime>,
}
```

4.5 Contradiction

```
struct Contradiction {
    id: ContradictionId,
    claim_a: ClaimId,
    claim_b: ClaimId,
    detected_at: DateTime,
    status: ConflictStatus,      // Detected, UnderReview, Resolved,
Accepted
    resolution: Option<Resolution>,
    resolved_by: Option<ResolverId>,
}
```

4.6 Artifact

```
struct Artifact {
    id: ArtifactId,
    artifact_type: ArtifactType, // EntityPage, DecisionLog, TaskPack,
Runbook, ArchMap
    content: String,
    version: u64,
    compiled_from: Vec<ClaimId>,
    citations: Vec<Citation>,
    health_score: f64,
    last_compiled: DateTime,
    stale: bool,
}
```

4.7 Workspace

```
struct Workspace {
    id: WorkspaceId,
    name: String,
    owner: UserId,
    policies: Vec<Policy>,
    agents: Vec<AgentId>,
    sources: Vec<SourceConfig>,
}
```

---

5. Multi-Agent Safety System

Three layers:

Layer 1: Agent Identity & Permissions

Every agent has a registered identity with explicit permissions:

- Read/write permissions per claim type
- Maximum sensitivity level access
- Rate limits (max claims per hour)
- Maximum confidence (agents can't override high-trust claims)
- Trust tier: System, Verified, Standard, Sandbox

Layer 2: Quarantine Pipeline

Agent-written claims pass through:

1. **Trust check** — is this agent authorized for this claim type?
2. **Anomaly detection** — is this claim anomalous vs. the agent's history?
3. **Conflict check** — does this contradict high-trust claims?

Claims that fail any check enter a quarantine queue for review.

Anomaly signals:

- Confidence much higher than agent's historical average
- Contradicts many high-trust claims simultaneously
- Burst of claims in short period (injection pattern)
- Source doesn't match agent's registered access
- Sensitivity level exceeds agent's permission

Layer 3: Formal Belief Revision

Knowledge updates follow AGM postulates:

1. **Success:** New information is represented in the updated state
2. **Inclusion:** No hallucinated implications added
3. **Vacuity:** Non-contradictory claims are simply added
4. **Consistency:** Contradictions are explicitly tracked, not silently created
5. **Minimal change:** Remove as little as possible to accommodate new information

When contradictions are detected:

- Compare source recency, confidence, and trust level
- Superseded claims get `valid_until` set (preserved, not deleted)
- All affected artifacts flagged for recompilation
- Full revision provenance logged

Enables temporal queries: "What did we believe about X on date Y?"

---

6. Belief Revision Engine

Inspired by the "Graph-Native Cognitive Memory" paper (March 2026).

Resolution strategy for contradictions:

Signal	Weight	Example
Source recency	High	Newer PR > older doc
Source trust level	High	Verified > Unknown
Claim confidence	Medium	0.95 > 0.70
Corroboration count	Medium	3 independent sources > 1
Author authority	Low	Team lead > bot

Auto-resolution: When signals clearly favor one claim (>80% weighted score), resolve automatically. Set the losing claim's `valid_until`, flag artifacts for recompile.

Human-required resolution: When signals are ambiguous (<80%), create a Contradiction record with status `UnderReview`. Surface in the contradiction resolution UI (paid feature). The human chooses: supersede A, supersede B, both valid (different contexts), or merge.

---

7. Access Control Model

Two-tier memory model:

- **Shared knowledge** — all workspace agents can read, write requires permission
- **Private memory** — per-agent, not visible to other agents

Sensitivity fence:

- Public → any agent
- Internal → workspace agents only
- Confidential → named agents only
- Restricted → human approval required

Promote/demote flow: Agents can propose promoting private claims to shared. Claims go through the quarantine pipeline before entering shared knowledge.

---

8. Product Layers

Open Source (Community)

- Compiler engine (parse, extract, link, compile)
- Core types and IR
- Kuzu + LanceDB embedded storage
- CLI: `root compile`, `root serve`, `root health`
- Basic MCP server

- File/Git/Web source parsers
- Basic artifact generation
- BYOK extraction (bring your own LLM key)

Paid Platform Only

- Continuous compilation (always-on, scheduled refresh)
- Contradiction resolution UI (human-in-the-loop)
- Source connectors (Slack, Jira, Linear, Confluence)
- Multi-agent safety engine (quarantine, anomaly, audit)
- Team workspaces + RBAC
- Knowledge health dashboard
- Policy engine (sensitivity, retention, access rules)
- Hosted compilation (no BYOK needed)
- Enterprise: SSO, VPC, compliance, custom ontologies

Principle: Open-source the compiler so anyone can try it. Keep the continuous, collaborative, safe layer paid.

---

9. Pricing Model

Hybrid: base subscription + included compile credits + overage.

	Community	Pro ($19/mo)	Team ($349/mo)	Enterprise
Deployment	Local CLI	Cloud	Cloud	Cloud / VPC / On-prem
Compilation	BYOK, manual	500 credits/mo	5,000 credits/mo	Unlimited
Sources	File/Git/Web	3 connected	50 connected	Unlimited
Workspaces	Local only	1	10	Unlimited
Refresh	Manual	Daily	Hourly	Real-time
Dashboard	CLI output	Health dashboard	+ Contradiction UI	+ Policy engine
Collaboration	Single user	Single user	RBAC + audit trail	SSO / SAML
Connectors	File/Git/Web	File/Git/Web	+ Slack/Jira/Linear	+ Custom
Support	Community	Email	Priority	Dedicated
Overage	N/A	$0.02/credit	$0.015/credit	Custom

What is a compile credit? 1 compile credit = 1 source document compiled through the full pipeline (parse → extract → link → compile → verify). A 500-file repo uses ~500 credits on first compile. Incremental recompilation of changed files uses 1 credit per changed file. This maps directly to LLM extraction cost + compute, making it predictable for both users and the platform.

Pricing rationale:

- Pro $19 enters below or at competitor starting prices (Supermemory $19, Letta $20, Zep $25)

- Team $349 reflects real cost of hosting compilation, connectors, and safety engine at scale
- Enterprise is custom due to high variance in needs
- Priced on things users feel: workspaces, sources, refresh frequency — not internal units

---

10. Competitive Positioning

All claims verified against official docs/sites as of April 2026.

Competitor	What they do	What ThinkingRoot does differently
Mem0	Memory compression engine. User/agent/session memory. Graph support. 100K+ devs. Strong DX.	Mem0 stores and retrieves compressed memories. ThinkingRoot compiles knowledge into verified artifacts with full provenance. Different primitive: memories vs. source-locked claims.
Zep/Graphiti	Temporal knowledge graph with bi-temporal tracking. 24.6K stars. Supports incremental episode ingestion and graph updates.	Graphiti is a temporal graph engine. ThinkingRoot is a full compilation pipeline that produces artifacts, not just graph queries. Adds verification CI, contradiction resolution UI, and multi-agent safety.
Cognee	Knowledge engine with ontology mapping. 38+ data types. Graph + vector. Enterprise-focused.	Cognee is closed-architecture enterprise. ThinkingRoot is open-core with a Rust engine anyone can inspect, extend, and embed.
Supermemory	Memory API + user profiles + RAG. Markets memory graph, extractors, connectors, and org-wide projects.	Overlapping capabilities in extraction and graph construction. ThinkingRoot's differentiator is the compilation pipeline (compress, deduplicate, resolve contradictions) and continuous verification.
Letta	Stateful agents with memory blocks. Portable agent memory. Growing platform.	Letta is agent-centric — memory belongs to an agent. ThinkingRoot is knowledge-centric — compiled knowledge is shared across agents, humans, and tools.
LangMem	Agent-controlled memory with any storage backend. Works with LangGraph natively, supports BaseStore abstraction.	LangMem gives agents memory tools. ThinkingRoot gives agents a pre-compiled knowledge base. Agent decides what to remember vs. system compiles all knowledge proactively.

Positioning statement: "Other tools store memories. ThinkingRoot compiles knowledge. Your agents read a verified, deduplicated, source-cited brief instead of re-processing raw documents every session."

No numeric claims (token reduction, speed) until benchmarks are published post-MVP.

---

11. Go-to-Market

Phase 1: Developer Adoption (Months 1-6)

Launch sequence:

- Weeks 1-2: Ship CLI + Python SDK (`pip install thinkingroot`, `cargo install thinkingroot`)
- Weeks 3-4: Ship MCP Server (any agent connects to ThinkingRoot)
- Month 2: Ship GitHub Action (auto-recompile on push)
- Month 3: Ship VS Code extension (browse compiled knowledge inline)
- Month 4: Public launch (Hacker News, Product Hunt)

Killer demo:

```
$ root ./my-repo
  ThinkingRoot compiled 847 files in 3.2s
  Knowledge Health: 91%
  ├── 1,247 claims extracted
  ├── 89 entities identified
  ├── 312 relations mapped
  ├── 7 contradictions found (3 auto-resolved)
  └── 23 artifacts generated
```

Phase 2: Platform Revenue (Months 6-12)

- Month 6: ThinkingRoot Cloud beta
- Month 7: Dashboard (knowledge explorer, health scores)
- Month 8: Contradiction Resolution UI
- Month 9: Team workspaces + collaboration
- Month 10: Slack, Linear, Jira connectors
- Month 11: Agent marketplace
- Month 12: Enterprise pilot program

Phase 3: Enterprise (Months 12-24)

- SSO / SAML, SOC2 / HIPAA compliance
- Dedicated compilation infra
- Custom ontologies and extractors
- Air-gapped deployment option
- SLA with health score guarantees

11.1 Personal Use Case — "Second Brain" Mode

ThinkingRoot doubles as a personal knowledge compiler — like Obsidian, but the compiler organizes for you.

What a personal user can ingest

Category	Examples	What ThinkingRoot extracts
Notes	Ideas, journal, thoughts, drafts	Claims, linked entities, topics
Articles/Bookmarks	Web pages, saved articles, blog posts	Key claims, cited to source URL
Code	Repos, snippets, side projects	Architecture, dependencies, decisions
Documents	PDFs, slides, spreadsheets, papers	Facts, figures, relationships
Conversations	Chat exports, meeting notes, voice memos	Decisions, action items, context
Social saves	Twitter bookmarks, Reddit saves, HN favorites	Insights, linked to topics
Media	Screenshots, images with text	OCR → claims (future phase)

How it organizes (graph, not folders)

Obsidian requires manual `[[links]]`, tags, and folders. ThinkingRoot auto-compiles:

```
User dumps:
  "Read this article about RAG"
  "Met John, he works on vector DBs at Pinecone"
  "Project idea: knowledge compiler"
  "This PDF about memory architectures"

ThinkingRoot auto-compiles into:

  Entities:
  ├── John (Person) → works at Pinecone, met 2026-04-08
  ├── Pinecone (Company) → vector DB, John works here
  ├── RAG (Concept) → linked to 3 articles, 1 project idea
  ├── Knowledge Compiler (Project) → your idea, linked to 12 sources
  └── Memory Architectures (Topic) → 1 PDF, 4 claims

  Artifacts:
  ├── entity/john.md — everything known about John, with sources
  ├── topic/rag.md — compiled summary of all RAG knowledge
  ├── project/knowledge-compiler.md — all related ideas + sources
  └── weekly-brief.md — what's new this week, auto-generated
```

Brain analogy mapping

Human Brain	ThinkingRoot Equivalent
Sensory input	Parse (raw data ingestion)
Short-term memory	Recent claims, not yet linked
Hippocampus (linking)	Link stage (entity resolution, connections)

Human Brain	ThinkingRoot Equivalent
Long-term memory	Compiled knowledge graph
Recall	Query / artifact generation
Forgetting	Staleness detection, confidence decay
"Wait, didn't I read the opposite?"	Contradiction detection + belief revision
Sleep consolidation	Compile stage (compress, deduplicate, strengthen links)

Comparison to existing tools

	Obsidian	Notion	ThinkingRoot
Organization	Manual tags/folders/links	Manual databases/pages	Automatic entity/claim graph
Connections	You create `[[links]]`	You create relations	Auto-detected from content
Staleness	You forget what's outdated	You forget what's outdated	CI flags stale claims
Contradictions	You don't notice them	You don't notice them	Auto-detected and surfaced
Summaries	You write them	You write them	Compiled automatically
Agent-readable	No	Limited	Yes, via MCP/SDK
Search	Keyword/filename	Keyword/filter	Semantic + graph traversal
Data ownership	Local files (good)	Cloud (their servers)	Local files (good)

Personal storage model

```
~/.thinkingroot/
├── sources/          # raw ingested data (cached originals)
├── graph/            # Kuzu embedded graph DB (~50MB for 10K claims)
├── vectors/          # LanceDB embeddings (~100MB for 10K claims)
├── artifacts/        # compiled markdown files (human-readable)
├── config.toml       # sources, refresh settings, LLM key
└── health.json       # latest health score
```

Everything local. Everything yours. No cloud required. No account needed.

Personal user workflow

```
# Dump anything into ThinkingRoot
root add ./my-notes/
root add https://interesting-article.com
root add ~/Downloads/research-paper.pdf
root add ~/Desktop/meeting-notes.md

# Compile your knowledge
root compile

# Ask questions
root query "what do I know about vector databases?"
# Returns: compiled brief citing your notes + articles + PDF

# Browse your compiled brain
root serve    # local MCP server — your AI agent reads your brain

# See what you're forgetting
root health   # flags stale notes, contradictions, gaps
```

Why this matters for adoption

The personal use case is the ultimate onboarding funnel:

1. Developer tries `root add` with their notes → instant value
2. They see compiled artifacts → "this is better than my Obsidian"
3. They connect their coding agent via MCP → "my agent knows everything I know"
4. They bring it to work → team adoption → paid tier

Personal use is free forever. It runs locally with zero dependencies. It's the gateway to team and enterprise adoption.

---

12. Technical Stack

Layer	Technology	Why
Compiler engine	Rust	50-100x faster than Python parsers. Speed is the moat.
Code parsing	tree-sitter	Industry standard, 50+ language grammars
Markdown parsing	pulldown-cmark	Rust standard, MIT
Embedded graph DB	Kuzu	SQLite of graph DBs. Zero-config, embeddable, MIT.
Hosted graph DB	Neo4j or FalkorDB	Enterprise-grade for cloud platform
Embedded vector DB	LanceDB	Rust-native, serverless, Apache 2.0

Layer	Technology	Why
Local embeddings	fastembed-rs	No API dependency for self-host
Python bindings	PyO3	The Polars/Ruff standard for Rust→Python
Template engine	Tera	Jinja2-compatible, for artifact generation
Serialization	serde + MessagePack	Fast binary IR
Async runtime	tokio	Rust standard
Cloud API	Axum	Fastest Rust web framework
Dashboard	Next.js 15 + TypeScript + Tailwind	Standard modern web stack
Auth	Clerk or Auth.js	Fast to integrate
Metadata DB	PostgreSQL	Users, billing, workspace metadata
Job queue	Redis streams	Compilation job queue
Object storage	S3-compatible	Raw source cache
Billing	Stripe	Standard
Early deployment	Docker + Fly.io or Railway	Fast iteration
Scale deployment	Kubernetes	Enterprise requirement

Self-host principle: `pip install thinkingroot` gives you: Rust engine + Kuzu graph + LanceDB vectors + fastembed embeddings + Tera templates. Zero Docker containers. Zero external services. Zero API keys for basic use (BYOK for LLM extraction).

---

13. Repository Structure

```
thinkingroot/
├── Cargo.toml                    # Rust workspace root
├── pyproject.toml                # Python SDK build config
├── package.json                  # Monorepo tooling (turbo)
│
├── crates/                       # Rust crates (the engine)
│   ├── thinkingroot-core/              # Core types: Claim, Entity,
Relation, etc.
│   ├── thinkingroot-parse/             # Stage 1: Parsers (markdown,
code, chat, pdf, git)
│   ├── thinkingroot-extract/           # Stage 2: Claim/entity/relation
extraction (Rust + LLM)
│   ├── thinkingroot-link/              # Stage 3: Entity resolution,
contradiction detection, belief revision
│   ├── thinkingroot-compile/           # Stage 4: Compilation targets
(entity pages, task packs, etc.)
│   ├── thinkingroot-verify/            # Stage 5: Memory CI (staleness,
```

```
consistency, poisoning, coverage)
│   ├── thinkingroot-graph/            # Graph storage abstraction (Kuzu,
Neo4j, LanceDB)
│   ├── thinkingroot-serve/            # Stage 6: REST API (Axum) + MCP
Server + query engine
│   ├── thinkingroot-safety/           # Multi-agent safety (registry,
quarantine, anomaly, access control)
│   └── thinkingroot-cli/              # CLI binary
│
├── sdk/
│   ├── python/                  # Python SDK (PyO3 bindings)
│   └── typescript/              # TypeScript SDK (REST API wrapper)
│
├── platform/                    # Cloud platform (Layer 3, paid)
│   ├── api/                     # Axum API server (auth, billing,
workspace management)
│   ├── web/                     # Next.js dashboard (explorer, health,
contradictions, settings)
│   └── workers/                 # Background jobs (compiler-worker,
source-watcher, verifier)
│
├── integrations/
│   ├── github-action/           # GitHub Action
│   ├── vscode/                  # VS Code extension
│   └── connectors/              # Source connectors (GitHub, Slack,
Linear, Jira, Confluence)
│
├── docs/                        # Documentation
│   ├── getting-started.md
│   ├── concepts/                # Claims, compilation, verification,
safety
│   └── api-reference/
│
└── tests/
    ├── fixtures/                # Test knowledge bases
    ├── benchmarks/              # Performance benchmarks
    └── integration/             # End-to-end compilation tests
```

---

14. Build Sequence

Phase 1: Core Engine (Weeks 1-8)

- Weeks 1-2: `thinkingroot-core` (types, IR)
- Weeks 3-4: `thinkingroot-parse` (markdown, code, git parsers)
- Weeks 5-6: `thinkingroot-extract` (claim extraction with LLM)
- Week 7: `thinkingroot-link` (entity resolution, contradiction detection)
- Week 8: `thinkingroot-compile` (entity pages, task packs)
- **Milestone:** `root ./repo` works end-to-end

Phase 2: Serve & Verify (Weeks 9-14)

- Weeks 9-10: `thinkingroot-graph` (Kuzu integration)
- Week 11: `thinkingroot-serve` (REST API + MCP server)
- Week 12: `thinkingroot-verify` (staleness, contradictions)
- Week 13: `thinkingroot-cli` (polish, error handling)
- Week 14: Python SDK via PyO3
- **Milestone:** Full open-source release on GitHub

Phase 3: Safety & SDK (Weeks 15-20)

- Weeks 15-16: `thinkingroot-safety` (agent registry, quarantine)
- Week 17: Belief revision engine
- Week 18: TypeScript SDK
- Week 19: GitHub Action
- Week 20: VS Code extension
- **Milestone:** Public launch (HN, Product Hunt)

Phase 4: Cloud Platform (Weeks 21-32)

- Weeks 21-24: Platform API (Axum, auth, billing)
- Weeks 25-28: Dashboard (Next.js, knowledge explorer)
- Weeks 29-30: Source connectors (Slack, Linear, Jira)
- Weeks 31-32: Continuous compilation workers
- **Milestone:** ThinkingRoot Cloud beta

Phase 5: Enterprise (Weeks 33-48)

- Weeks 33-36: SSO, audit logs, compliance
- Weeks 37-40: Custom ontologies, enterprise connectors
- Weeks 41-44: Air-gapped deployment option
- Weeks 45-48: Enterprise pilot program
- **Milestone:** First enterprise contracts

---

15. Success Metrics

Phase 1 Metrics (Developer Adoption)

Metric	Month 3	Month 6
Time to first artifact	< 60s	< 30s
Weekly active repos	200	1,000
Median token reduction	Measure & publish	> 85% (verified)
Artifact refresh latency	< 10s incremental	< 5s
Weekly active MCP connections	50	500

Phase 2 Metrics (Platform Revenue)

Metric	Month 9	Month 12
Paying workspaces	50	200
MRR	$5K	$30K
Free → Pro conversion	3%	5%
Contradictions resolved/workspace/week	Track	> 5
Net revenue retention	Track	> 110%

Phase 3 Metrics (Enterprise)

Metric	Month 18	Month 24
Enterprise pilots	3	10
Enterprise ACV	$30K	$50K+
MRR	$100K	$500K+

North star metric: Contradictions resolved per workspace per week. High resolution rate = system finding real problems + humans trusting it + continuous value.

---

16. Risks & Mitigations

Risk	Severity	Mitigation
Compilation quality too low — users don't trust compiled artifacts	Critical	Start with narrow scope (code repos only). Measure accuracy obsessively. Always show source citations so users can verify. Never hide the raw source.
Cold start problem — compilation takes too long before value	High	Design the CLI demo to compile a repo in < 60 seconds. First artifacts must be useful immediately, not after days of ingestion.
Competitor adds compilation — Mem0/Zep/Supermemory copy the feature	Medium	The Rust engine + formal belief revision + safety layer create technical moat. Compilation quality is an engineering problem that takes years to master, not a feature toggle.
LLM extraction costs too high — BYOK users face high API bills	Medium	Use small, fast models (Haiku-class) for extraction. Cache extraction results aggressively. Incremental compilation means you only re-extract changed sources.
Open-source community doesn't adopt	Medium	Ship a compelling CLI demo first. Focus on developer experience. Make `root compile` as satisfying as `cargo build`. Publish honest benchmarks.

Risk	Severity	Mitigation
Enterprise sales cycle too long	Medium	Enterprise is Phase 3, not Phase 1. Revenue starts from Pro ($19) and Team ($349) tiers. Enterprise pilots begin only after platform is proven.
Memory poisoning in production	High	The safety engine (quarantine, anomaly detection, belief revision) is a first-class design constraint, not an afterthought. Ship safety from Phase 3, before multi-agent write access is widely used.
Graph storage scaling	Medium	Start with Kuzu (embedded) for self-host. Cloud uses Neo4j/FalkorDB which scale to millions of nodes. Design the graph abstraction layer so backends are swappable.

Appendix A: Key Research References

Papers that inform the design (verified titles and dates):

- **MemMachine** (April 2026) — ground-truth-preserving multi-tier memory architecture
- **Springdrift** (April 2026) — persistent append-only memory with case-based reasoning
- **ByteRover** (April 2026) — agent-curated hierarchical Context Trees
- **MemFactory** (March 2026) — unified framework for memory-augmented agents, GRPO fine-tuning
- **Graph-Native Cognitive Memory** (March 2026) — formal belief revision with property graphs
- **SuperLocalMemory V3.3** (April 2026) — biologically-inspired forgetting with Fisher-Rao quantization
- **Memory Intelligence Agent** (April 2026) — Manager-Planner-Executor with bidirectional memory conversion
- **Deep Researcher Agent** (April 2026) — Two-Tier Constant-Size Memory (~5K chars)
- **Poison Once, Exploit Forever / eTAMP** (April 2026) — cross-session memory poisoning via environment
- **No Attacker Needed** (April 2026) — unintentional cross-user contamination in shared memory
- **Collaborative Memory** (May 2025) — multi-user private + shared memory with access control

Appendix B: Competitor Landscape Summary

Product	Model	Stars/Users	Approach	What they lack
Mem0	Hosted API + OSS	100K+ devs	Memory compression	No compilation, no verification
Zep/Graphiti	OSS + hosted	24.6K stars	Temporal knowledge graph	No artifact generation, no compilation
Cognee	Closed cloud	Enterprise	Ontology + graph + vector	Not open-source, no CI
Supermemory	Hosted + OSS	Growing	Memory + RAG + profiles	No compilation pipeline, no formal verification

Product	Model	Stars/Users	Approach	What they lack
Letta	OSS + hosted	Growing	Stateful agent memory	Per-agent only, no shared compilation
LangMem	OSS (LangChain)	LangGraph users	Agent-controlled memory	Framework-native, no proactive compilation

This spec was designed collaboratively on 2026-04-08. All competitive claims verified against official docs/sites as of that date. No numeric performance claims are made without published benchmarks.